

few exceptions [73, 92]. Evidence from small-scale, constrained randomized controlled experiments demonstrates a productivity increase ranging from 21% [102] to 56% [94], as measured by task completion time. Field experiments conducted at Microsoft, Accenture, and Cisco report similar numbers (from 22% [40] to 36% [91]). The productivity increase estimated from open-source projects on observational data is similar and sometimes lower: a DiD design comparing Python and R packages estimates a 17.82% increase in new releases among Python packages after Copilot availability [111]—without clear knowledge of which packages used Copilot. Another study of proprietary Copilot backend data estimates only a 6.5% increase in project-level productivity, as measured by the number of accepted pull requests [102]. A more general study using a neural classifier to identify AI-generated code on GitHub shows moving to 30% AI use raises quarterly commits by 2.4% [43]. Studies point to various mechanisms causing the productivity increase, such as how LLM adoption increases work autonomy [63] and helps iterative development tasks (e.g., bug fixing) [111].

Although the productivity gains are promising, there are also increasing concerns around the trustworthiness of LLM-generated code. For example, it is well-known that LLMs may generate code with security vulnerabilities [21, 53, 70, 78, 93], performance regressions [75], code smells [101], and outdated APIs [69, 109]. On the other hand, evidence regarding the complexity of LLM-generated code compared to humans is inconclusive [39, 81, 87]. The LLM trustworthiness problem becomes more complicated with humans in the loop. For example, prior controlled experiments report mixed results on whether developers write more or less secure code with the help of LLMs [88, 95, 97], and studies often suggest heterogeneous treatment effects of LLMs on developers of different skill levels [40, 42, 94, 102]. While prior work points to many mechanisms by which adopting LLMs may affect software quality, their findings are typically derived from benchmark analyses [e.g., 93] or developer opinions [e.g., 81]. We are unaware of any prior studies that systematically investigated project-level quality outcomes in the wild after LLM adoption, let alone those that used rigorous causal inference techniques (the closest being the study by Yeverchayahu et al. [111] discussed above).

Recently, there has been an increasing interest in the application of *LLM agents*—LLMs with the capability to autonomously utilize external resources and tools—to software engineering [61, 67, 77]. A popular application scenario is an *LLM agent assistant* within a code editor, in which LLMs are allowed to inspect/edit project files, conduct web searches, and execute shell commands to fulfill prompts provided by developers. At the time of writing, there are several production-ready code editors with built-in LLM agent assistants, such as Cursor [8], VS Code [19], Windsurf [20], Tabnine [17], and Cline [5], with the extreme beginning to shift away from IDEs entirely and switching to command-line or web interfaces, such as Claude Code [4] and OpenHands [12]. These agentic tools are seeing rapid adoption among developers, as evidenced also in our data for Cursor in Figure 2. From the gray literature, we see extremely optimistic estimates of the productivity boost from LLM agent assistants: for example, developers self-report multi-fold productivity increases in a Reddit post [15], orders of magnitude larger than any empirical estimates for prior LLM tools. However, a recent controlled study with human participants shows that developers may

be overoptimistic and that adopting LLM agent assistants does not make them faster in real-world open-source development tasks [27]. To the best of our knowledge, empirical evidence regarding the impact of LLM agent assistants on *long-term project-level outcomes*, especially *software quality outcomes*, is still lacking.

Our contribution is two-fold. First, our DiD design looks at the additional project-level productivity gain, if any, from using a modern *agentic* coding assistant (Cursor) *relative to the state-of-the-practice* (likely a mixture of human-written code and code generated by earlier-generation AI tools). Second, we provide a comprehensive analysis of the impact of adopting Cursor on code quality, which is the first to the best of our knowledge, and highlight potential velocity-quality trade-offs and their complex interactions.

3 Research Design and Methods

We estimate the *causal* effects of adopting Cursor on *development velocity* and *code quality*, both of which are considered important project outcomes, are commonly measured in prior research, and are closely tied to perceived overall project productivity [38, 52, 89]. We start by building a dataset with: (1) repositories adopting Cursor at different times and (2) comparable repositories that never adopted Cursor (Section 3.1). Then, we define our specific outcomes of interest and additional covariates (Section 3.2). The estimation of Cursor’s causal effect on these outcomes is enabled by a difference-in-differences (DiD) design with staggered adoption [29] (Section 3.3): Under the assumption that similar repositories would, on average, evolve similarly in the absence of Cursor adoption (i.e., the *parallel trend assumption*), later-adopters and never-adopters can effectively serve as a quasi-experimental comparison group for earlier-adopters while accounting for observable covariates and macro trends (e.g., open-source repositories overall getting more or less active over time). Finally, since the results from DiD suggest *interactions* between development velocity and software quality, as also indicated in prior work [28, 38], we fit dynamic panel generalized method of moments (GMM) models [22] to support our interpretation of the DiD results (Section 3.4).

3.1 Data Collection

3.1.1 The Cursor IDE. Cursor [8] is an AI-powered IDE built as a VS Code fork with agentic capabilities integrated into the development workflow. Unlike earlier code completion tools, Cursor’s agentic mode enables an autonomous, goal-directed AI workflow: The agent can navigate entire codebases, infer project architecture across multiple files, make multi-file edits, run terminal commands, execute tests, and iteratively debug code, with humans mainly serving as supervisors rather than traditional coders. Using this workflow, developers can rely entirely on AI for feature implementation, refactoring, test generation, documentation, and bug fixing within their native development environment. They can choose between frontier models from OpenAI, Anthropic, and Google, either via Cursor’s built-in service or their own API keys.

We chose Cursor as our main study case for two reasons. First, our preliminary exploration found widespread and growing adoption compared to competing tools, providing sufficient statistical power for causal inference. Second, Cursor is among the earliest to allow optional configuration files (e.g., `.cursorrules`) in the git